Software Requirements Specification

ChatWave — A WhatsApp-style Messaging App (v0)

Kodex Cohort · Docker / DevOps Phase · Reference Standard

> *How to read this document: Sections 2 (Scope) and 4 (Functional Requirements) are the contract. If something is not written here, we are not building it. The first thing we decided was what we are NOT building — read Section 2.2 before anything else.*

1. Introduction

1.1 Purpose

This document defines what the ChatWave v0 messaging application must do, and just as importantly, what it must not do. It is the single source of truth for the build. “Done” means “matches this specification” — not “I felt finished.”

1.2 Why we wrote this before coding

Code is the most expensive way to discover you misunderstood a requirement. A sentence is cheap to change; a built feature is not. Writing this first forces every scope decision to happen on paper — now — instead of at 11pm in week three when it breaks the data model.

1.3 Product overview

ChatWave is a real-time, one-to-one chat application. A registered user can find another user, open a conversation, and exchange text messages that are delivered instantly while both are online and persisted so they are visible on the next login. The product is deliberately scoped to the smallest version that is still recognisably “a chat app.”

2. Scope

2.1 In scope (v0)

- **User accounts** — register, log in, log out (JWT-based authentication).
- **Find a user** — search for another registered user by username or email.
- **One-to-one conversations** — a private conversation between exactly two users.
- **Real-time messaging** — text messages delivered live via WebSocket (Socket.io) when the recipient is online.
- **Message persistence** — messages are stored and reloaded when a user reopens a conversation.
- **Online / offline status** — show whether the other user is currently connected.

2.2 Out of scope (v0) — read this first

These are deliberately excluded. Deciding what we are NOT building is the real engineering judgment. Anything below is a future version or a stretch goal, never a v0 surprise:

- Group chats (many-to-many).
- Media messages — images, audio, video, files, voice notes.
- Read receipts (the blue ticks) and “typing…” indicators.
- End-to-end encryption.
- Push notifications (mobile / browser).
- Message editing, deleting, forwarding, or replies/threads.

- Voice and video calling.
- Offline message composition / send queue.

Note: “Online/offline status” is IN scope; “read receipts” and “last seen” are OUT. Know the difference — they feel similar but have very different costs.

3. Users & Assumptions

Actor	Description
Guest	An unauthenticated visitor. Can only register or log in.
Registered user	An authenticated user. Can search users, open conversations, and send/receive messages.

Assumptions: users access the app from a modern browser; a user is logged in on one device at a time; the network is generally available (we do not engineer for offline use in v0).

4. Functional Requirements

Each requirement is numbered and testable. “The system shall…” means it is verifiable: you can point at the running app and say yes or no.

4.1 Authentication

ID	Requirement	Priority
FR-1	The system shall let a guest register with a unique username, email, and password.	Must
FR-2	The system shall reject registration if the username or email already exists.	Must
FR-3	The system shall let a registered user log in and receive a JWT.	Must
FR-4	The system shall protect all messaging endpoints and socket connections behind authentication.	Must
FR-5	The system shall let a user log out, invalidating the client session.	Must

4.2 Finding users & conversations

ID	Requirement	Priority
FR-6	The system shall let a user search for other users by username or email.	Must
FR-7	The system shall let a user open a one-to-one conversation with another user.	Must
FR-8	The system shall reuse the existing conversation if one already exists between the two users.	Must
FR-9	The system shall show a user the list of their existing conversations, most recent first.	Should

4.3 Messaging

ID	Requirement	Priority
FR-10	The system shall let a user send a text message within an open conversation.	Must
FR-11	The system shall deliver a message to the recipient in real time when they are online.	Must
FR-12	The system shall persist every message so it is visible when the conversation is reopened.	Must
FR-13	The system shall display messages in chronological order with sender and timestamp.	Must
FR-14	The system shall store messages sent to an offline user and deliver them on next login.	Must

4.4 Presence

ID	Requirement	Priority
FR-15	The system shall indicate whether the other user in a conversation is currently online.	Should
FR-16	The system shall update online/offline status in real time as users connect and disconnect.	Should

5. Non-Functional Requirements

ID	Requirement
NFR-1	Real-time delivery: a message should reach an online recipient in under ~1 second on a normal connection.
NFR-2	Ordering: messages within a conversation must always display in the order they were sent.
NFR-3	Security: passwords are stored hashed (never plaintext); all protected routes require a valid JWT.
NFR-4	Portability: the entire app must run via a single docker compose up with no manual host setup.
NFR-5	Configuration: all secrets and connection strings come from environment variables, never hard-coded.
NFR-6	Statelessness: the backend must hold no in-memory state that prevents running more than one container instance.

NFR-4 to NFR-6 are the DevOps-phase requirements. They are why this project exists: the app must be containerized, configuration-driven, and deployable to ECS behind a load balancer.

6. Data Model (Entities)

Three core entities. This is the shape of the data, not the final schema — you will refine it, but you may not add entities that serve out-of-scope features.

Entity	Key fields
User	_id, username (unique), email (unique), passwordHash, createdAt
Conversation	_id, participants [two user ids], lastMessageAt, createdAt
Message	_id, conversationId, senderId, text, createdAt, delivered (bool)

7. Interface Contract

7.1 REST endpoints

Endpoint	Purpose
POST /api/auth/register	Create a new user account.
POST /api/auth/login	Authenticate and return a JWT.
GET /api/users?search=	Search users by username or email.
GET /api/conversations	List the current user's conversations.
POST /api/conversations	Open (or reuse) a conversation with another user.
GET /api/conversations/:id/messages	Load message history for a conversation.

7.2 Socket.io events

Event	Direction & purpose
message:send	Client → server: send a new message in a conversation.
message:new	Server → client: deliver an incoming message in real time.
presence:online	Server → client: a user came online.
presence:offline	Server → client: a user went offline.

8. Acceptance Criteria ("Done")

v0 is done when, and only when, all of the following are demonstrably true:

- Two users on two browsers can register, log in, find each other, and exchange messages that appear instantly.
- Closing and reopening a conversation shows the full message history in order.
- A message sent to an offline user is delivered when that user logs back in.
- Each user can see whether the other is online.
- The whole system starts with a single docker compose up on a clean machine.
- The same image is deployed and reachable on ECS behind a load balancer, with WebSockets working.

Change rule: any new feature must first be added to this document (and its scope section) before a line of code is written for it. The spec leads; the code follows.